

16 · Practice Problems

How to use this chapter. Cover the solution, write your own query on paper or in SQL Developer, then compare. Every SQL answer below was **executed against the sample tables** and the result shown is real. Problems are grouped by topic and marked ★ (warm-up), ★★ (typical interview) or ★★★ (stretch). They follow the patterns of the college assignment — filtering, joins, grouping, ranking, dates, LIKE / REGEXP , CASE — on a small **employee / department** database.

The two tables (dates are shown in ISO form here; Oracle displays them as DD-MON-RR by default — "today" is **16-AUG-2025**):

dept

dept_id	dept_name	location
10	Management	Chennai
20	Engineering	Bengaluru
30	Sales	Mumbai
40	HR	Pune

emp

emp_id	emp_name	job	manager_id	salary	comm	dept_id	hire_date
101	Asha	CEO	NULL	90000	NULL	10	2015-01-10
102	Bala	Manager	101	70000	NULL	20	2016-03-15
103	Chitra	Developer	102	50000	NULL	20	2018-07-01
104	Dinesh	Developer	102	50000	NULL	20	2019-02-20
105	Esha	Tester	102	40000	NULL	20	2020-11-05
106	Farhan	Manager	101	65000	NULL	30	2017-05-30
107	Gita	Sales Rep	106	35000	5000	30	2021-01-12
108	Hari	Sales Rep	106	35000	3000	30	2022-06-18
109	Isha	Intern	103	20000	NULL	20	2024-01-08
110	Jai	Consultant	101	45000	NULL	NULL	2023-04-01

emp.manager_id refers to another employee (the boss), emp.dept_id to dept . **Jai has no department** and **HR has no employees** — deliberately, so joins and NOT IN behave interestingly.

Part A — Filtering, sorting, functions

A1 ★ — Engineers earning at least 45,000, highest paid first.

```

SELECT emp_name, job, salary
FROM emp
WHERE dept_id = 20 AND salary >= 45000
ORDER BY salary DESC, emp_name;

```

Result

emp_name	job	salary
Bala	Manager	70000
Chitra	Developer	50000
Dinesh	Developer	50000

A2 ★ — Employees whose name starts with a vowel (A, E, I, O, U).

```

SELECT emp_name FROM emp
WHERE emp_name LIKE 'A%' OR emp_name LIKE 'E%' OR emp_name LIKE 'I%'
      OR emp_name LIKE 'O%' OR emp_name LIKE 'U%'
ORDER BY emp_name;

```

Result

emp_name
Asha
Esha
Isha

Regex alternative: `WHERE REGEXP_LIKE(emp_name, '^[AEIOU]')` .

A3 ★ — Employees hired between 2019 and 2022 (inclusive), with the date formatted.

```

SELECT emp_name, TO_CHAR(hire_date, 'DD-MON-YYYY') AS hired_on
FROM emp
WHERE hire_date BETWEEN DATE '2019-01-01' AND DATE '2022-12-31'
ORDER BY hire_date;

```

Result

emp_name	hired_on
Dinesh	20-FEB-2019
Esha	05-NOV-2020
Gita	12-JAN-2021
Hari	18-JUN-2022

A4 ★ — Total pay (salary plus commission, treating a missing commission as 0) for everyone in Sales.

```
SELECT emp_name, salary, comm, salary + NVL(comm, 0) AS total_pay
FROM emp
WHERE dept_id = 30
ORDER BY emp_id;
```

Result

emp_name	salary	comm	total_pay
Farhan	65000	NULL	65000
Gita	35000	5000	40000
Hari	35000	3000	38000

A5 ★ — Names with at least 6 characters.

```
SELECT emp_name, LENGTH(emp_name) AS len FROM emp WHERE LENGTH(emp_name) >= 6 ORDER BY emp_name;
```

Result

emp_name	len
Chitra	6
Dinesh	6
Farhan	6

A6 ★★ — Years of service (whole years) as of today, longest-serving first.

```
SELECT emp_name, hire_date,
       TRUNC(MONTHS_BETWEEN(SYSDATE, hire_date) / 12) AS years_of_service
FROM emp
ORDER BY hire_date
FETCH FIRST 5 ROWS ONLY;
```

Result

emp_name	hire_date	years_of_service
Asha	2015-01-10	10
Bala	2016-03-15	9
Farhan	2017-05-30	8
Chitra	2018-07-01	7
Dinesh	2019-02-20	6

A7 ★★ — Classify salaries into bands and count the employees in each.

```

SELECT CASE WHEN salary >= 60000 THEN 'High'
           WHEN salary >= 40000 THEN 'Medium'
           ELSE 'Low' END AS band,
COUNT(*) AS employees
FROM emp
GROUP BY CASE WHEN salary >= 60000 THEN 'High'
           WHEN salary >= 40000 THEN 'Medium'
           ELSE 'Low' END
ORDER BY MIN(salary) DESC;

```

Result

band	employees
High	3
Medium	4
Low	3

(`GROUP BY` repeats the whole `CASE` — Oracle doesn't accept the alias `band` there.)

A8 ★★ — A friendly message for each employee, with the hire date in single quotes: Dear Gita, you were hired on '12-JAN-2021'.

```

SELECT 'Dear ' || emp_name || ', you were hired on ' ||
       TO_CHAR(hire_date, 'DD-MON-YYYY') || ' ' AS message
FROM emp
WHERE emp_id IN (101, 107)
ORDER BY emp_id;

```

Result

message
Dear Asha, you were hired on '10-JAN-2015'.
Dear Gita, you were hired on '12-JAN-2021'.

`' '` inside a string is one literal quote; `''''` is a string containing exactly one quote (Chapter 09).

Part B — Joins

B1 ★ — Every employee with the name of their department; keep employees who have none.

```

SELECT e.emp_name, d.dept_name
FROM emp e
LEFT JOIN dept d ON e.dept_id = d.dept_id
ORDER BY e.emp_id;

```

Result

emp_name	dept_name
Asha	Management
Bala	Engineering
Chitra	Engineering
Dinesh	Engineering
Esha	Engineering
Farhan	Sales
Gita	Sales
Hari	Sales
Isha	Engineering
Jai	NULL

B2 ★★ — Departments that have no employees.

```
SELECT d.dept_name
FROM   dept d
LEFT   JOIN emp e ON e.dept_id = d.dept_id
WHERE  e.emp_id IS NULL;
```

Result

dept_name
HR

B3 ★★ — Each employee with their manager's name; the CEO shows No manager .

```
SELECT e.emp_name AS employee, COALESCE(m.emp_name, 'No manager') AS manager
FROM   emp e
LEFT   JOIN emp m ON e.manager_id = m.emp_id
ORDER BY e.emp_id;
```

Result

employee	manager
Asha	No manager
Bala	Asha
Chitra	Bala
Dinesh	Bala
Esha	Bala
Farhan	Asha
Gita	Farhan
Hari	Farhan

employee	manager
Isha	Chitra
Jai	Asha

B4 ★★ — Every manager and the number of people who report directly to them.

```
SELECT m.emp_name AS manager, COUNT(*) AS direct_reports
FROM emp e
JOIN emp m ON e.manager_id = m.emp_id
GROUP BY m.emp_name
ORDER BY direct_reports DESC, manager;
```

Result

manager	direct_reports
Asha	3
Bala	3
Farhan	2
Chitra	1

B5 ★★ — The salary cost of each department, including departments with no staff (show 0).

```
SELECT d.dept_name, NVL(SUM(e.salary), 0) AS total_salary
FROM dept d
LEFT JOIN emp e ON e.dept_id = d.dept_id
GROUP BY d.dept_name
ORDER BY total_salary DESC;
```

Result

dept_name	total_salary
Engineering	230000
Sales	135000
Management	90000
HR	0

B6 ★★★ — Colleagues of Gita: the other people in her department.

```
SELECT c.emp_name
FROM emp g
JOIN emp c ON c.dept_id = g.dept_id AND c.emp_id <> g.emp_id
WHERE g.emp_name = 'Gita'
ORDER BY c.emp_name;
```

Result

emp_name
Farhan
Hari

Part C — Aggregation

C1 ★ — Head-count and average salary per department.

```
SELECT dept_id, COUNT(*) AS headcount, ROUND(AVG(salary)) AS avg_salary
FROM emp
GROUP BY dept_id
ORDER BY dept_id;
```

Result

dept_id	headcount	avg_salary
10	1	90000
20	5	46000
30	3	45000
NULL	1	45000

C2 ★★ — Departments with more than two employees.

```
SELECT dept_id, COUNT(*) AS headcount
FROM emp
GROUP BY dept_id
HAVING COUNT(*) > 2
ORDER BY dept_id;
```

Result

dept_id	headcount
20	5
30	3

C3 ★★ — Job titles held by more than one person.

```
SELECT job, COUNT(*) AS people
FROM emp
GROUP BY job
HAVING COUNT(*) > 1
ORDER BY job;
```

Result

job	people
Developer	2
Manager	2
Sales Rep	2

C4 ★★ — Highest, lowest and spread of salary in each department.

```
SELECT dept_id, MAX(salary) AS highest, MIN(salary) AS lowest, MAX(salary) - MIN(salary) AS spread
FROM emp
WHERE dept_id IS NOT NULL
GROUP BY dept_id
ORDER BY dept_id;
```

Result

dept_id	highest	lowest	spread
10	90000	90000	0
20	70000	20000	50000
30	65000	35000	30000

C5 ★★ — How many employees earn a commission, and how much is paid in total?

```
SELECT COUNT(comm) AS earners, SUM(comm) AS total_commission FROM emp;
```

Result

earners	total_commission
2	8000

C6 ★★★ — Per department: number of developers, number of people paid above 40,000, and total salary — in one query.

```
SELECT d.dept_name,
       SUM(CASE WHEN e.job = 'Developer' THEN 1 ELSE 0 END) AS developers,
       SUM(CASE WHEN e.salary > 40000 THEN 1 ELSE 0 END) AS paid_over_40k,
       NVL(SUM(e.salary), 0) AS total_salary
FROM dept d
LEFT JOIN emp e ON e.dept_id = d.dept_id
GROUP BY d.dept_name
ORDER BY d.dept_name;
```

Result

dept_name	developers	paid_over_40k	total_salary
Engineering	2	3	230000
HR	0	0	0

dept_name	developers	paid_over_40k	total_salary
Management	0	1	90000
Sales	0	1	135000

Part D — Subqueries

D1 ★ — Employees earning more than the company average.

```
SELECT emp_name, salary FROM emp WHERE salary > (SELECT AVG(salary) FROM emp) ORDER BY salary DESC;
```

Result

emp_name	salary
Asha	90000
Bala	70000
Farhan	65000

D2 ★★ — The second-highest salary.

```
SELECT MAX(salary) AS second_highest FROM emp WHERE salary < (SELECT MAX(salary) FROM emp);
```

Result

second_highest
70000

D3 ★★ — The highest-paid employee(s) of each department.

```
SELECT d.dept_name, e.emp_name, e.salary
FROM emp e JOIN dept d ON e.dept_id = d.dept_id
WHERE e.salary = (SELECT MAX(salary) FROM emp WHERE dept_id = e.dept_id)
ORDER BY d.dept_name;
```

Result

dept_name	emp_name	salary
Engineering	Bala	70000
Management	Asha	90000
Sales	Farhan	65000

D4 ★★ — Departments whose *average* salary is above the company average.

```
SELECT dept_id, ROUND(AVG(salary)) AS avg_salary
FROM emp
GROUP BY dept_id
HAVING AVG(salary) > (SELECT AVG(salary) FROM emp);
```

Result

dept_id	avg_salary
10	90000

D5 ★★ — People who work in a department with at least three employees.

```
SELECT emp_name, dept_id FROM emp
WHERE dept_id IN (SELECT dept_id FROM emp GROUP BY dept_id HAVING COUNT(*) >= 3)
ORDER BY dept_id, emp_name;
```

Result

emp_name	dept_id
Bala	20
Chitra	20
Dinesh	20
Esha	20
Isha	20
Farhan	30
Gita	30
Hari	30

D6 ★★ — Employees who manage at least one person (EXISTS).

```
SELECT m.emp_name
FROM emp m
WHERE EXISTS (SELECT 1 FROM emp e WHERE e.manager_id = m.emp_id)
ORDER BY m.emp_id;
```

Result

emp_name
Asha
Bala
Chitra
Farhan

D7 ★★★ — Employees who earn more than *every* Sales Rep (> ALL).

```
SELECT emp_name, salary
FROM emp
WHERE salary > ALL (SELECT salary FROM emp WHERE job = 'Sales Rep')
ORDER BY salary DESC;
```

Result

emp_name	salary
Asha	90000
Bala	70000
Farhan	65000
Chitra	50000
Dinesh	50000
Jai	45000
Esha	40000

> ALL means "greater than the **largest**" — the two Sales Reps earn 35,000, so anyone above 35,000 qualifies.

D8 ★★★ — Which department has the most developers? (subquery inside HAVING .)

```
SELECT d.dept_name, COUNT(*) AS developers
FROM emp e JOIN dept d ON e.dept_id = d.dept_id
WHERE e.job = 'Developer'
GROUP BY d.dept_name
HAVING COUNT(*) = (SELECT MAX(c) FROM (SELECT COUNT(*) AS c FROM emp WHERE job = 'Developer' GROUP BY dept_id));
```

Result

dept_name	developers
Engineering	2

Part E — Window functions

E1 ★★ — Rank employees by salary inside each department, showing how ties behave.

```
SELECT dept_id, emp_name, salary,
       RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk,
       DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS dense_rnk
FROM emp
WHERE dept_id IS NOT NULL
ORDER BY dept_id, rnk, emp_name;
```

Result

dept_id	emp_name	salary	rnk	dense_rnk
10	Asha	90000	1	1
20	Bala	70000	1	1
20	Chitra	50000	2	2
20	Dinesh	50000	2	2
20	Esha	40000	4	3
20	Isha	20000	5	4
30	Farhan	65000	1	1
30	Gita	35000	2	2
30	Hari	35000	2	2

E2 ★★ — The second-highest salary *in each department*.

```
SELECT DISTINCT dept_id, salary AS second_highest
FROM (SELECT dept_id, salary,
             DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS r
      FROM emp WHERE dept_id IS NOT NULL)
WHERE r = 2
ORDER BY dept_id;
```

Result

dept_id	second_highest
20	50000
30	35000

Department 10 has a single employee, so it has no second-highest — it simply doesn't appear.

E3 ★★ — Running total of the company payroll in hiring order.

```
SELECT emp_name, hire_date, salary,
       SUM(salary) OVER (ORDER BY hire_date) AS running_payroll
FROM emp
ORDER BY hire_date;
```

Result

emp_name	hire_date	salary	running_payroll
Asha	2015-01-10	90000	90000
Bala	2016-03-15	70000	160000
Farhan	2017-05-30	65000	225000
Chitra	2018-07-01	50000	275000
Dinesh	2019-02-20	50000	325000
Esha	2020-11-05	40000	365000

emp_name	hire_date	salary	running_payroll
Gita	2021-01-12	35000	400000
Hari	2022-06-18	35000	435000
Jai	2023-04-01	45000	480000
Isha	2024-01-08	20000	500000

E4 ★★ — Each employee's salary as a percentage of their department's total.

```
SELECT emp_name, dept_id, salary,
       ROUND(100 * salary / SUM(salary) OVER (PARTITION BY dept_id), 1) AS pct_of_dept
FROM   emp
WHERE  dept_id = 30
ORDER BY emp_id;
```

Result

emp_name	dept_id	salary	pct_of_dept
Farhan	30	65000	48.1
Gita	30	35000	25.9
Hari	30	35000	25.9

E5 ★★★ — Keep only the earliest-hired person for each (department, job) combination.

```
SELECT emp_name, dept_id, job, hire_date
FROM   (SELECT emp_name, dept_id, job, hire_date,
              ROW_NUMBER() OVER (PARTITION BY dept_id, job ORDER BY hire_date) AS rn
        FROM   emp)
WHERE  rn = 1
ORDER BY dept_id, job;
```

Result

emp_name	dept_id	job	hire_date
Asha	10	CEO	2015-01-10
Chitra	20	Developer	2018-07-01
Isha	20	Intern	2024-01-08
Bala	20	Manager	2016-03-15
Esha	20	Tester	2020-11-05
Farhan	30	Manager	2017-05-30
Gita	30	Sales Rep	2021-01-12
Jai	NULL	Consultant	2023-04-01

E6 ★★★ — Every second employee (2nd, 4th, 6th ...) by employee id.

```
SELECT emp_id, emp_name
FROM (SELECT emp_id, emp_name, ROW_NUMBER() OVER (ORDER BY emp_id) AS rn FROM emp)
WHERE MOD(rn, 2) = 0;
```

Result

emp_id	emp_name
102	Bala
104	Dinesh
106	Farhan
108	Hari
110	Jai

Part F — Classic interview one-liners

Question	Answer
Find duplicate values	<code>SELECT col, COUNT(*) FROM t GROUP BY col HAVING COUNT(*) > 1;</code>
N-th highest salary	<code>SELECT DISTINCT salary FROM (SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) r FROM emp) WHERE r = :n;</code>
Delete duplicate rows, keep one (Oracle)	<code>DELETE FROM t WHERE ROWID NOT IN (SELECT MIN(ROWID) FROM t GROUP BY dup_col1, dup_col2);</code>
Top 3 per group	<code>DENSE_RANK() OVER (PARTITION BY g ORDER BY x DESC)</code> in a subquery, <code>WHERE rn <= 3</code>
Rows in A but not in B	<code>SELECT ... FROM a MINUS SELECT ... FROM b;</code> or <code>NOT EXISTS</code>
Employees earning more than their manager	<code>SELECT e.emp_name FROM emp e JOIN emp m ON e.manager_id = m.emp_id WHERE e.salary > m.salary;</code>
Swap two values with CASE	<code>UPDATE t SET flag = CASE flag WHEN 'M' THEN 'F' WHEN 'F' THEN 'M' END;</code>
Count rows fast without COUNT(*) ?	there's no reliable alternative — <code>COUNT(*)</code> is optimised; <code>COUNT(1)</code> is identical
Employees with a name containing exactly two vowels	<code>WHERE REGEXP_COUNT(emp_name, '[aeiouAEIOU]') = 2</code>
Table with no primary key — de-duplicate with a window	<code>ROW_NUMBER() OVER (PARTITION BY cols ORDER BY ROWID) = 1</code>

Part G — PL/SQL practice

Assumed for the last problems: the `emp(emp_id, emp_name, salary, dept_id ...)` and `dept` tables above. All programs assume `SET SERVEROUTPUT ON;`

P1 ★ — Sum of the digits of a number ($n = 1234 \rightarrow 10$).

```
DECLARE
  n NUMBER := 1234;
  t NUMBER;
  s NUMBER := 0;
BEGIN
  t := n;
  WHILE t > 0 LOOP
    s := s + MOD(t, 10);    -- last digit
    t := TRUNC(t / 10);    -- drop it
  END LOOP;
  DBMS_OUTPUT.PUT_LINE('Sum of digits of ' || n || ' = ' || s);
END;
/
```

Sum of digits of 1234 = 10

P2 ★ — Reverse a number ($1234 \rightarrow 4321$). Same loop, build $r := r * 10 + \text{MOD}(t, 10);$:

```
DECLARE
  t NUMBER := 1234;
  r NUMBER := 0;
BEGIN
  WHILE t > 0 LOOP
    r := r * 10 + MOD(t, 10);
    t := TRUNC(t / 10);
  END LOOP;
  DBMS_OUTPUT.PUT_LINE('Reversed: ' || r);
END;
/
```

Reversed: 4321

P3 ★★ — Is a number prime? ($n = 17$.) Only test divisors up to the square root:

```

DECLARE
    n          NUMBER := 17;
    is_prime   BOOLEAN := (n > 1);
BEGIN
    FOR i IN 2 .. TRUNC(SQRT(n)) LOOP
        IF MOD(n, i) = 0 THEN
            is_prime := FALSE;
            EXIT;
        END IF;
    END LOOP;
    IF is_prime THEN
        DBMS_OUTPUT.PUT_LINE(n || ' is prime');
    ELSE
        DBMS_OUTPUT.PUT_LINE(n || ' is not prime');
    END IF;
END;
/

```

```
17 is prime
```

P4 ★★ — The first 8 Fibonacci numbers.

```

DECLARE
    a NUMBER := 0;
    b NUMBER := 1;
    c NUMBER;
    line VARCHAR2(200);
BEGIN
    FOR i IN 1 .. 8 LOOP
        line := line || a || ' ';
        c := a + b;
        a := b;
        b := c;
    END LOOP;
    DBMS_OUTPUT.PUT_LINE(line);
END;
/

```

```
0 1 1 2 3 5 8 13
```

P5 ★★ — A procedure that gives a percentage raise to one department and reports how many people were affected (implicit cursor).

```

CREATE OR REPLACE PROCEDURE give_raise (p_dept IN NUMBER, p_pct IN NUMBER DEFAULT 10) IS
BEGIN
    UPDATE emp
    SET    salary = salary * (1 + p_pct / 100)
    WHERE dept_id = p_dept;

    IF SQL%NOTFOUND THEN
        DBMS_OUTPUT.PUT_LINE('No employees in department ' || p_dept);
    ELSE
        DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' employees got a ' || p_pct || '% raise');
    END IF;
END give_raise;
/

EXEC give_raise(30, 5);

```

3 employees got a 5% raise

P6 ★★ — A function that converts marks to a grade (CASE), then use it in SQL.

```

CREATE OR REPLACE FUNCTION grade_of (p_marks IN NUMBER) RETURN VARCHAR2 IS
BEGIN
    RETURN CASE
        WHEN p_marks >= 90 THEN 'A'
        WHEN p_marks >= 75 THEN 'B'
        WHEN p_marks >= 60 THEN 'C'
        ELSE 'F'
    END;
END grade_of;
/

SELECT name, marks, grade_of(marks) AS grade FROM students;

```

P7 ★★ — With an explicit cursor, print each department and its head-count (including empty ones).

```

DECLARE
    CURSOR c IS
        SELECT d.dept_name, COUNT(e.emp_id) AS n
        FROM   dept d LEFT JOIN emp e ON e.dept_id = d.dept_id
        GROUP BY d.dept_name
        ORDER BY d.dept_name;
    rec c%ROWTYPE;
BEGIN
    OPEN c;
    LOOP
        FETCH c INTO rec;
        EXIT WHEN c%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(RPAD(rec.dept_name, 12) || ': ' || rec.n);
    END LOOP;
    CLOSE c;
END;
/

```

```

Engineering : 5
HR          : 0
Management  : 1
Sales       : 3

```

P8 ★★★ — A trigger that forbids salary cuts.

```

CREATE OR REPLACE TRIGGER trg_no_pay_cut
BEFORE UPDATE OF salary ON emp
FOR EACH ROW
WHEN (NEW.salary < OLD.salary)           -- no colon inside WHEN
BEGIN
    RAISE_APPLICATION_ERROR(-20010, 'Salary cannot be reduced for employee ' || :OLD.emp_id);
END;
/

UPDATE emp SET salary = 30000 WHERE emp_id = 106;
-- ORA-20010: Salary cannot be reduced for employee 106

```

P9 ★★★ — A package with a function and a procedure.

```

CREATE OR REPLACE PACKAGE emp_util IS
    FUNCTION annual_pay (p_id IN NUMBER) RETURN NUMBER;
    PROCEDURE list_dept (p_dept IN NUMBER);
END emp_util;
/

CREATE OR REPLACE PACKAGE BODY emp_util IS

    FUNCTION annual_pay (p_id IN NUMBER) RETURN NUMBER IS
        v_pay NUMBER;
    BEGIN
        SELECT salary * 12 + NVL(comm, 0) INTO v_pay FROM emp WHERE emp_id = p_id;
        RETURN v_pay;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN RETURN NULL;
    END annual_pay;

    PROCEDURE list_dept (p_dept IN NUMBER) IS
    BEGIN
        FOR r IN (SELECT emp_name, salary FROM emp WHERE dept_id = p_dept ORDER BY emp_id) LOOP
            DBMS_OUTPUT.PUT_LINE(r.emp_name || ' - ' || r.salary);
        END LOOP;
    END list_dept;

END emp_util;
/

SELECT emp_util.annual_pay(107) FROM dual;      -- 35000 * 12 + 5000 = 425000
EXEC emp_util.list_dept(30);

```

```

Farhan - 65000
Gita - 35000
Hari - 35000

```

P10 ★★★ — College-style multi-step exercises (orders / line items). Assumed schema: customers(custno, custname, address), items(itemno, itemname, rate), orders(orderno, orderdate, custno, shipdate, shipaddress), lineitems(orderno, itemno, qty, rate, discount).

1. **Insert a new row into orders and lineitems:** orderno = 1 + the highest order number; order date = yesterday; customer 103; ship date = 15 days after the order; the shipping address is the customer's address; the line item is item 4 at the **least rate** of the orders, quantity 2, discount 0. Hint: `SELECT NVL(MAX(orderno), 0) + 1 INTO ...; SYSDATE - 1; SYSDATE - 1 + 15; an INSERT ... SELECT for the address; MIN(rate) from lineitems`. Do both inserts in one block and `COMMIT` once.
2. **Display the item name and the number of units sold**, ignoring items with no sales. Hint: inner `JOIN` + `GROUP BY` + `SUM(qty)`.
3. **Insert a line item** for order 1003, item 4, quantity 1, rate = the item's current rate, discount 5 %. Hint: `INSERT ... SELECT rate FROM items WHERE itemno = 4`.

4. **Apply conditions to that insert:** the item must not already exist on the order; the order total so far must not cross 30,000; the order must have been placed in the last 4 days. *Hint: a PL/SQL block with a `SELECT COUNT(*)`, a `SUM(qty * rate)`, then `IF ... THEN INSERT ... ELSE RAISE_APPLICATION_ERROR`.*
 5. **Display the order amount placed by the first 5 customers.** *Hint: `FETCH FIRST 5 ROWS ONLY` or a `RANK()` subquery; `GROUP BY custno`.*
 6. **Change each item's rate: +10 %** if it has more than 5 orders **or** more than 25 units sold; **+5 %** if it was sold at a rate higher than its current rate. *Hint: a cursor `FOR` loop with `COUNT(DISTINCT orderno)` and `SUM(qty)`, or a single `UPDATE ... SET rate = CASE ...`.*
-

Quick self-check

- Can you explain **why** D7's `> ALL` and `> MAX` are the same, and why `NOT IN` in B2 would have failed?
- Can you rewrite any Part C query with a window function, and any Part E query with a subquery?
- For A7, why must `GROUP BY` repeat the `CASE` instead of using the alias?
- In P8, why is it `WHEN (NEW.salary ...)` but `:OLD.emp_id` in the body?